

Product master and affected services — Function Design (機能設計書)

004_DD-FN defines WI-006 server processing for [004_DD](#), the approved [004_DD-API](#) version 2, [004_DB](#) version 2 and [order/dashboard DB addendum](#). It covers REQ-049–REQ-054 and REQ-056–REQ-060. The approved WI-002–WI-004 function designs remain read-only; this file owns WI-006 deltas to their services. It is a design, not an implemented or tested service.

Document control (改版履歴)

Field	Value
Document ID	004_DD-FN
Work item	WI-006
System / subsystem	ProductionManagementAI / Product master, production orders and dashboard
Created / updated	Codex, 2026-09-29 / 2026-09-30
Version	3 — distinguish design-consistency review from implementation verification

Version	Change
1	Initial Product master service draft under plan revision 1
2	Reconcile product-unit immutability, retirement races, measured quantities and dashboard aggregates
3	Clarify that PostgreSQL collation, row-lock and error-path integration tests are implementation acceptance checks, not prerequisites for a design-only consistency review

Function catalog and layer boundaries

ID / existing method	Application operation	Reads / writes	Requirement
FN-027	ListProductsAsync , GetProductAsync	products plus bounded reference-existence reads	REQ-049, REQ-057
FN-028	CreateProductAsync	products insert	REQ-050, REQ-054
FN-029	UpdateProductAsync	Locked products row, production_orders existence, product update	REQ-051, REQ-057

ID / existing method	Application operation	Reads / writes	Requirement
FN-030	<code>RetireProductAsync</code>	Versioned <code>products</code> update	REQ-052, REQ-054
FN-031	Order-product eligibility and quantity validation	Locked <code>products</code> row, existing order create/edit	REQ-052, REQ-056, REQ-058–REQ-059
Existing <code>DashboardService.GetSnapshotAsync</code> / <code>IDashboardReader.ReadAsync</code>	Unit-aware Q2–Q5 read and mapper changes; retains existing dashboard function IDs	Read-only snapshot over orders/products	REQ-060

The API layer authenticates and enforces `Admin` / `Operator`, checks JSON shape/raw quantity token and maps service outcomes to the codes in 004_DD-API. The Application layer owns business rules and transaction coordination. Domain types use exact `decimal` for order quantity; Infrastructure uses EF Core/Npgsql, a parameterized row-lock query, raw SQL for the existing dashboard reader, constraint/unique-violation translation and owner-run migrations. No new ORM, external service or architectural boundary is introduced. Cancellation is propagated; canceled requests are not counted as an unexpected failure.

Shared data and result mapping

Input or projected value	Function use	Mapping / rule
<code>ProductMasterItem.unitLocked</code>	FN-027–FN-030	<code>EXISTS</code> order by <code>product_id</code> ; computed at read time, never stored. It is informational and can become stale; FN-029 rechecks under lock.
<code>products.xmin</code>	FN-027, FN-029, FN-030	<code>version</code> in API; mismatch becomes <code>PRODUCT_STALE</code> . Locking a row alone does not replace version comparison.
<code>products.unit</code> , <code>is_active</code>	FN-029/FN-031	Unit mutation blocked after any order reference; a new/changed order selection needs active state.
Raw JSON <code>quantity</code> token → decimal	FN-031	API rejects string, exponent, >3 written fractional digits and invalid range before parsing; service rechecks positivity, max, scale and discrete-unit integrality after product lock.
<code>production_orders.quantity numeric</code>	FN-031/dashboard	Read/write exact decimal; no <code>double</code> / <code>float</code> conversion, no integer DTO.
Dashboard Q2 (<code>bucket</code> , <code>unit</code> , <code>count</code> , <code>quantity</code>)	Existing dashboard mapper	Bucket <code>orderCount</code> is sum of counts across unit rows; <code>unitQuantities</code> holds one decimal subtotal per present unit. Never sum the subtotals.

The module calls no third-party HTTP service. Shared utilities are the existing plant clock, order-number issuer, EF context/repository and Problem Details mapper; no new global utility is required. API field catalogs stay in 004_DD-API.

FN-027 — Search and load products

Inputs: validated `q`, `state`, `page` or product UUID. **Returns:** `PagedResult<ProductMasterItem>`, one `ProductMasterItem`, or `NotFound`. No mutation.

1. Trim `q` and reject >100 characters; allow only `all`, `active`, `retired` and `page` ≥ 1 . The API converts invalid inputs to 400 field errors before querying.
2. Apply parameterized case-insensitive SKU/name search and state filter over **all** products, including retired. Count filtered rows; order by SKU then ID; fetch at most 20 page rows. The search uses the configured PostgreSQL collation, with no invented SKU-normalization rule.
3. Project `id`, `sku`, `name`, `unit`, `drawing_number`, `is_active`, `updated_at_utc`, `xmin` and `unitLocked` using `EXISTS` on `production_orders.product_id` for each returned product. The indexed existence check does not fetch order payloads. Detail uses the same projection by UUID; missing ID yields `NotFound`.
4. Preserve the existing `GET /api/products` full array and SKU ordering; extend its projection with `unit` and `isActive`. This endpoint remains the order picker/filter source and includes retired rows.

The list's `unitLocked` can change after response; it is for rendering, not authorization. A missing unit after migration is an integrity failure, not an implicit 個. Search/read queries use no new index until measured cost justifies one.

FN-028 — Create product

Input: normalized Product master create request. **Returns:** persisted `ProductMasterItem` or validation/SKU conflict.

1. The API rejects unknown fields and malformed JSON. Application trims SKU/name/drawing number; blank drawing becomes null. Validate required fields, 50/200/100 character limits and the seven-unit allowlist. A request may not set ID, state, version or timestamps.
2. Optionally precheck `lower(sku)` for a friendly conflict; the DB unique index `ux_products_sku_lower` remains final authority. Do not use a divergent case comparison in memory.
3. Insert one active row with server-generated UUID and UTC timestamps. Save once; map the named unique-index violation from a concurrent create to 409 `PRODUCT_SKU_CONFLICT` with `errors.sku`. Do not translate unrelated SQL errors into duplicate conflicts.
4. Return the saved row/version with `isActive=true` and `unitLocked=false`. No order or sequence row is written. Validation or uniqueness failure leaves no partial product.

FN-029 — Edit product under reference lock

Input: product UUID, required current SKU/name/unit/version and optional drawing number. **Returns:** updated `ProductMasterItem`, `NotFound`, validation or conflict.

1. Validate request shape and field lengths. Begin a transaction and acquire `SELECT ... FOR UPDATE` on the target `products` row **before** querying order references. Missing ID gives `NotFound`. The product-row lock is held through commit/rollback.
2. Compare submitted `version` with current `xmin`; stale version returns 409 `PRODUCT_STALE` and rolls back. Compare SKU exactly with stored SKU; changed SKU gives 400 `errors.sku`. Neither check changes data.
3. If requested unit differs, run indexed `EXISTS (SELECT 1 FROM production_orders WHERE product_id = @id)` **after** obtaining the product lock. A reference gives 409 `PRODUCT_UNIT_LOCKED` with `errors.unit` and rollback. If unit is unchanged, name/drawing number may be corrected even when referenced or retired.
4. Update only `name`, allowed `unit`, `drawing_number`, `updated_at_utc`. Preserve ID, SKU, `created_at_utc`, `is_active`. Save with EF optimistic `xmin` condition; translate a concurrency exception to `PRODUCT_STALE`, then commit and return the new version and recomputed `unitLocked`.

Every order create/changed-product write acquires `FOR SHARE` on the same product before inserting or changing the order and holds it through commit. PostgreSQL makes that lock conflict with the product update, so no order can appear between step 3's reference test and step 4's save. A new order referencing an unreferenced product after an edit commits freezes the new unit from that point on. A retired but unreferenced product may change unit; retirement alone is not a unit lock.

FN-030 — Retire product

Input: UUID and current version only. **Returns:** retired `ProductMasterItem`, `NotFound`, `invalid-already-retired` or stale conflict.

1. Validate body, begin transaction, and load/lock product row for update. Missing ID gives `NotFound`; stale `xmin` gives `PRODUCT_STALE`; already-retired gives 400 validation.
2. Set `is_active=false` and `updated_at_utc` to the server clock. Preserve unit, SKU, name, drawing number, ID and all orders. Save with optimistic version check; commit and return the new version/derived `unitLocked`.
3. A concurrent new/changed order selection holds `FOR SHARE` through its commit. If it locked first, its valid order commits before retirement. If retirement locked first, the later selection sees inactive state and returns `PRODUCT_INACTIVE`. No invalid selection commits after a completed retirement.

There is no restore or physical delete operation. A referenced product may be retired; old orders remain readable and a Draft edit may retain its unchanged retired selection.

FN-031 — Order selection and exact quantity

Order path	Product eligibility	Quantity behavior
Create	Product exists and is active at locked read; otherwise <code>errors.productId</code>	Validate against locked unit before issuing order sequence.
Draft edit, changed product ID	New product exists and is active at locked read	Revalidate against the new unit; a previously valid decimal may become invalid.
Draft edit, unchanged product ID	Existing product may be retired	Validate edited quantity against that product's immutable unit.
Non-Draft edit	Existing product/quantity must remain unchanged under existing state rule	Preserve stored exact quantity and joined unit; status, due-date and notes rules remain as approved.

Create sequence: API validates raw numeric token; Application validates shape and product-independent range. Begin the existing order-number transaction. Acquire `SELECT ... FOR SHARE` on selected product, check existence/active state, then validate quantity against its unit. If invalid, roll back **before** issuing a sequence. If valid, issue the order number, insert order, save and commit; the product lock lasts through commit. This replaces the old pretransaction `ProductExistsAsync` check as the authoritative eligibility check. The existing order-number and due-date behavior is preserved.

Draft update sequence: validate request shape; begin transaction; load existing order and check version/status rules; determine whether product ID changes; acquire `FOR SHARE` on the selected product and read its unit/active state. An unchanged retired product is allowed; a changed retired product is rejected. Validate exact decimal quantity, apply allowed order fields, save with the existing optimistic version check and commit. A 400 `PRODUCT_INACTIVE` or `QUANTITY_UNIT_INVALID` rolls back, retaining the previous order. A stale order version keeps the existing conflict behavior. Where the order remains non-Draft, enforce its existing product/quantity immutability before any save; a product lock is unnecessary for unchanged immutable fields.

The API raw-token check rejects `1.2340` even if a numeric parser would preserve or normalize it differently. The service uses decimal scale no greater than 3, `quantity > 0`, `quantity <= 999999999`, and `quantity == truncate(quantity)` for discrete units. `kg / m` permit `0.001` and other positive values up to three fractional digits. The DB `numeric` CHECKs defend range/scale, while the service owns the cross-table discrete-unit rule. An exact decimal is returned with its joined product unit; no floating-point intermediate is allowed.

Existing dashboard service — Q1–Q6 adaptation

`DashboardService.GetSnapshotAsync` still reads the clock once and derives `today`, `asOf` and plant-time windows from it. `IDashboardReader.ReadAsync` still executes Q1–Q6 (Q3 twice) inside one read-only `REPEATABLE READ` transaction with parameterized Npgsql commands. It does not call Product master endpoints or open a write transaction.

Query	Reader change	Mapper / API result
Q1	None	Existing status counts.
Q2	Group active orders by existing due bucket and <code>products.unit</code> ; read <code>count(*)</code> , <code>sum(quantity)</code> per pair	Build exactly 10 buckets; sum counts per bucket into <code>orderCount</code> , keep each unit's decimal subtotal in <code>unitQuantities</code> sorted by approved unit order; empty bucket gets <code>[]</code> . Never sum unlike quantities.
Q3a/Q3b	Join <code>products.unit</code> for each attention order; keep window total and first-10 order	Map decimal quantity and <code>product.unit</code> ; group <code>total</code> remains a count.
Q4	Group by product ID/SKU/name/unit; count active orders and sum decimal quantity within product ; sort count descending, SKU ascending; limit 10	<code>activeOrderCount</code> drives rank; <code>openQuantity</code> and <code>product.unit</code> label one physical unit.
Q5	Keep completed week/month <code>count(*)</code> , rate and lead-time inputs; remove mixed-product quantity sums	<code>completedThisWeek</code> / <code>completedThisMonth</code> contain <code>orderCount</code> and <code>from</code> only.
Q6	None	Existing 12-week completion count.

The Q2 reader returns at most 70 unit/bucket rows. Missing unit/product after migration is an integrity failure and the whole snapshot returns the existing generic failure, not a fabricated 個 . `numeric` sums are mapped to exact decimal values; no `long` / `double` cast. The ten workload bucket dates and twelve trend weeks follow the approved 003_DB windows. Workload bars use order counts, and top products omit zero-active-order products. Dashboard fields removed by 004_DD-API are not serialized. No live query has run in this design phase.

Persistence, privileges, errors and instrumentation

Concern	Function rule
Database access	EF Core/Npgsql for product/order writes; parameterized SQL for <code>FOR UPDATE</code> / <code>FOR SHARE</code> and the existing dashboard reader. No string-built user input in SQL.
Runtime role	<code>pmai_app</code> retains order grants and its existing product <code>SELECT</code> ; gains product <code>INSERT</code> and column-scoped <code>UPDATE</code> (<code>name</code> , <code>unit</code> , <code>drawing_number</code> , <code>is_active</code> , <code>updated_at_utc</code>) per 004_DB; no product <code>DELETE</code> or DDL. Owner role runs migrations.
Error translation	Named SKU unique violation → <code>PRODUCT_SKU_CONFLICT</code> ; stale product version → <code>PRODUCT_STALE</code> ; changed referenced unit → <code>PRODUCT_UNIT_LOCKED</code> ; inactive new order selection → <code>PRODUCT_INACTIVE</code> ; unit-specific amount → <code>QUANTITY_UNIT_INVALID</code> . Other database faults stay generic. Exact HTTP/field keys are in 004_DD-API.
Retry	No automatic retry of product edits, retirements, order writes or a dashboard failure. A user-triggered reload gets fresh version/lock state; serialization/deadlock failures are surfaced as generic failure without replaying a mutation.

Use a `ProductionManagementAI.Products` ActivitySource/Meter for `Product.List`, `Product.Get`, `Product.Create`, `Product.Update`, `Product.Retire` and `Product.UnitLockCheck`. Extend existing `ProductionOrder.Create / Update` spans with bounded validation outcome (`valid`, `inactive`, `quantity_invalid`, `not_found`) and the dashboard span with `unit_aggregation` outcome; do not create a span per bucket/row. Counters: `pmai.products.requests` tagged by bounded operation/outcome, `pmai.orders.unit_validation` by bounded outcome, and existing dashboard-loaded counter by `success / error`. No SKU, name, drawing number, raw decimal text, cookie, full body or user-supplied search appears in logs or metrics; product ID is not a metric label. Unexpected exceptions are logged structurally with operation, trace ID and safe outcome, while the API returns generic Problem Details. Follow existing trace sampling and avoid high-cardinality status/tag values.

Migration dependency and verification boundary

The owner-run 004_DB migration backfills reviewed units for 30 seed IDs, preserves IDs/FKs, removes model-managed `HasData` without `DeleteData`, adds the case-insensitive SKU index and product grants. The separate order migration converts positive integers to exact `numeric` under a scheduled write pause. Both must finish and validate before the decimal-aware API and dashboard are deployed; old/new binaries do not overlap during the schema cutover. A failed index build or migration is handled as in the DB documents, and a fractional order prevents an automatic data-preserving Down conversion to integer. This design phase has run no migration, live SQL, backend test or telemetry capture.

Unresolved business decisions: none. The design-consistency review checks that the specified SKU comparison, row-lock and error paths agree across the approved BD/DB/API/DD documents and have traceable test cases. Implementation must verify the configured PostgreSQL collation's `lower(sku)` behavior and those lock/error paths with integration tests before delivery is accepted. [004_DD-SPD](#) and the [affected-screen DD addendum](#) are approved client-design inputs to that review.